

Does Compositional Refinement Strictly Extend Heuristic Specification Inference? A Measurement Study

Brendan Edmonds, Mark Utting

UQ Cyber, School of Electrical Engineering and Computer Science
The University of Queensland, St Lucia QLD 4067, Australia
{b.edmonds, m.utting}@uq.edu.au

Abstract—Heuristic specification inferers compute a per-method specification from local syntactic evidence. When a method calls another, the callee’s specification is typically incorporated by single-pass propagation: at each call site, the callee’s preconditions (substituted for actual arguments) are appended to the caller’s precondition set. A natural question is whether this single pass is *compositional* in the sense that matters — whether a second, top-down weakest-precondition (WP) refinement pass would discover additional preconditions that the isolated pass missed. We implement the second pass for an existing JML inferer (a 414-line analyser that walks each method’s body in reverse-topological order, lifts callee specifications through enclosing branch guards, and iterates to a fixpoint on cyclic strongly-connected components) and measure its effect on Apache Commons Lang and Commons IO. The result is unambiguous: the second pass refines 36–44% of all methods in the cache and adds 2,001 / 5,763 new precondition clauses respectively. The dominant shape (53–57%) is branch-conditional implications of the form `guard ==> caller_obligation`, which the single-pass propagation does not produce because it lacks the control-flow context the second pass walks. The single pass does *not* subsume the compositional pass on real Java; we report which clause shapes drive the gap and where the residual is.

Index Terms—JML, specification inference, compositional reasoning, weakest precondition, Java, static analysis

I. INTRODUCTION

A heuristic specification inferer for Java produces, for each method, a set of Java Modeling Language (JML) clauses inferred from local syntactic evidence: a null-dereference of a parameter inside the body becomes a `requires \p != null` clause; an array read at index `i` becomes `requires 0 <= i && i < a.length`; and so on. Recent work has shown that such specifications, even when heuristically produced, materially improve downstream test generation [1] and can be embedded in compiled artefacts with full lossless roundtrip [2]. Both of those results depend on the spec set being *rich enough* to constrain downstream tools.

The dominant strategy for incorporating cross-method information in such an inferer is *single-pass propagation*: while inferring method `m`, the analyser looks up each callee’s cached specification, substitutes the actual arguments at the call site, and appends the substituted callee preconditions to `m`’s precondition set. This is fast and order-independent (provided the callee is inferred first), and it is what the inferer of

[1] already does via its `InterproceduralAnalyzer`. A natural question is whether this single pass captures everything a *genuinely compositional* analysis would: that is, whether a second, top-down weakest-precondition (WP) refinement pass, walking each method’s body with the callee specifications substituted at each call site, would discover any additional clauses that the single pass missed.

The question matters operationally. If the answer is no, the second-pass refactoring (and the associated implementation, testing, and maintenance cost) is unjustified; the simpler single-pass architecture is good enough. If the answer is yes, downstream consumers of inferred specifications — test generators, verifiers, LLM-based tooling — see a measurably richer spec set when the second pass runs.

In this paper, we implement the second pass and answer the question empirically on real Java code. We address the following research questions:

- **RQ1.** Does a top-down WP-style compositional refinement pass strictly extend the precondition set produced by an established single-pass propagation, or is the single pass already sufficient?
- **RQ2.** If the second pass adds clauses, which structural shapes (null checks, range bounds, branch-conditional implications, disjunctive obligations) dominate the additions? The answer characterises *why* the single pass falls short.
- **RQ3.** What is the runtime cost of the second pass at corpus scale, and how does it interact with cyclic strongly-connected components (mutual recursion) in the call graph?

We answer RQ1–RQ3 on Apache Commons Lang 3.14.0 (3,494 methods) and Apache Commons IO 2.13.0 (2,111 methods), the same libraries used by the predecessor work [1], [2]. The principal findings are summarised in Table I and analysed in Section V; in short, the second pass refines 36–44% of methods in the specification cache and adds thousands of new precondition clauses, with branch-conditional implications (`guard ==> obligation`) as the dominant shape.

Contributions. In summary, this paper makes the following contributions:

- A 414-line `CompositionalAnalyzer` that implements the top-down WP refinement pass over a populated specification cache, supporting per-call-site argument substitution, enclosing-guard lifting, polymorphic-dispatch disjunction over candidate callees, and bounded fixpoint iteration on cyclic strongly-connected components.
- An empirical measurement on two real-world Java libraries (5,605 inferred methods in total) showing that the second pass refines 36–44% of methods in the cache and adds 7,764 new precondition clauses overall; the single-pass strategy is *not* subsuming.
- A shape-categorisation of the newly-added preconditions showing that 53–57% are branch-conditional implications (guard $\Rightarrow$ obligation) — the shape the single pass structurally cannot produce, because it lacks the control-flow context the WP walk supplies.
- Two real bugs found in the analyser by running it at scale: a `lastIndexOf('.')` in the polymorphic-candidate lookup that failed on parameter types containing dots (e.g. `java.lang.String`), and a related off-by-one in the call-site resolver. Both are fixed in the artefact accompanying this paper.

Paper organisation. Section II reviews single-pass propagation and the gap it leaves. Section III describes the second-pass algorithm precisely. Section IV describes the implementation. Section V reports the measurement. Section VI discusses what the result means for single-pass inferrer architectures and for the downstream tools that consume inferred specifications. Section VII addresses the threats to validity, Section VIII positions the work against related literature, and Section IX concludes.

II. BACKGROUND AND MOTIVATION

A. Heuristic JML inference and single-pass propagation

The inferrer of [1] is a heuristic JML producer for Java: a library of pattern-matchers over the `JavaParser` AST that emits a `MethodSpecification` per method declaration. Twelve specialist analysers (precondition, postcondition, loop-invariant, assignable, nullability, overflow, etc.) cooperate via a `SpecificationCache` indexed by simple-class method signature.

Cross-method propagation is handled by an `InterproceduralAnalyzer` component invoked during isolated inference. When method m contains a call $n(args)$, this analyser looks up n 's cached specification, substitutes formal parameters with actual arguments, and appends the substituted preconditions to m 's precondition set. The pass is single-pass: each method is visited once in topological order over the call graph; the callee's spec is read once and added at the call site without further transformation. We refer to this as the *isolated* strategy throughout the paper, since each method's analysis treats callees as black boxes whose contracts are pulled in atomically.

B. Where the single-pass strategy is blind

The single-pass propagation is the right strategy for the simple case: a precondition that the callee requires *unconditionally* must hold at the call site, and conjoining it into the caller's set is sound. But the strategy is structurally blind to two classes of information.

Branch-conditional obligations. If the caller m has a body fragment

```
if (g) {
    n(args); // n requires p
}
```

then m 's precondition is not $p[args/params]$ but rather $g \Rightarrow p[args/params]$ — the obligation must hold only on the path where the call is reached. Single-pass propagation either appends $p[args/params]$ unconditionally (overconstraining m on paths where the call does not fire) or skips the substitution to be safe (missing the obligation entirely on the guarded path). Neither is correct.

Polymorphic-dispatch obligations. If m calls $x.n()$ where x 's declared type has multiple implementations of n , the runtime dispatch may pick any candidate. A sound caller precondition must hold for *at least one* candidate: it is the disjunction ($p_1 \parallel p_2 \parallel \dots \parallel p_k$) of substituted candidate preconditions. Single-pass propagation, which resolves to a single nominal callee, cannot produce this disjunction.

C. What a second-pass compositional analysis would add

The classic weakest-precondition calculus of Dijkstra [3] addresses both shapes structurally. Walking a method's body in WP fashion accumulates the guard context at each program point, so a call site reached only on g contributes $g \Rightarrow p$; a polymorphic call site contributes the disjunction over candidates by construction.

Implementing a full WP transformer for arbitrary Java would require an SMT decision procedure and a precise operational semantics for every language construct, which is the domain of full verification tools (KeY, OpenJML's SC mode) rather than heuristic inferers. A *scaffold* that walks the body in a WP-flavoured order, lifts each call site's substituted callee specification through enclosing branch guards, and recognises polymorphic candidates is, however, implementable as a 400-line pass over the AST. That is what this paper does.

D. What we measure, and what we do not

We do not measure whether the second-pass specifications are *semantically correct* against the code's intent — that is the inferrer's responsibility, not the compositional pass's, and it is the question of [1]. The compositional pass is a structural transformation: given the cached callee specs (whatever their semantic quality), it produces a new spec for the caller. We measure whether that new spec contains clauses the single pass did not.

The downstream impact on LLM-based test generation (whether the richer specs measurably improve mutation-killing rate) is the subject of a planned follow-up. The present article is scoped to the structural claim — does the second pass add anything to a real-world single-pass baseline — because the answer matters even if the downstream impact is small (it tells us the architectural distinction is real) and because the answer is empirically tractable in a way the downstream comparison is not.

III. THE COMPOSITIONAL REFINEMENT PASS

The pass takes as input a populated `SpecificationCache`, a call graph over the same methods, and a map from method signature to `MethodDeclaration` (the AST). It mutates the cache in place.

A. Driver

The driver iterates strongly-connected components (SCCs) of the call graph in reverse topological order, so callees are refined before callers. For each SCC:

- *Singleton SCC (no recursion)*: refine the single method once.
- *Cyclic SCC (mutual recursion)*: refine each method in the SCC; repeat until no method gains a new clause or until a configurable fixpoint cap (default: 16 iterations) is reached. The cap exists to bound runtime on pathological graphs; we log a warning when it fires.

B. Per-method refinement

For each method `m`, the refiner walks every `MethodCallExpr` in `m`'s body and, for each call site, performs the following sequence:

- 1) **Resolve the callee.** Match the call's simple name against keys in the cache to find a candidate `n`. A full type-resolving harness is not required for the present measurement; mismatches caused by overloading are documented as a measurement limitation (Section VII).
- 2) **Substitute arguments.** For each precondition of `n`, replace each formal parameter name with the actual argument expression. Substitution is regex-based with word boundaries so a formal `s` does not clobber a substring `this.size`. Side-effecting actuals (`++`, `--`) reject the substitution as unsound.
- 3) **Lift through enclosing guards.** Walk the AST upward from the call expression and accumulate the conjunction of any `if`-condition or ternary-condition whose body contains the call. If guards are present, the substituted precondition becomes `(guard) ==> (substituted)`. Calls in the `else` branch contribute `!(guard)`. Side-effecting guards are rejected to preserve soundness.
- 4) **Detect polymorphic dispatch.** If the resolved callee has overriding implementations in subclasses (looked

up via the call graph's class-hierarchy index), collect every candidate's substituted preconditions and emit the disjunction `(p_1 || p_2 || ... || p_k)`. The disjunction is sound: at least one candidate's contract must hold for the dispatch to be safe.

- 5) **Conjoin into the caller's set.** Add the lifted clause to `m`'s precondition list if it is not already present.

C. Termination-aware postcondition propagation

The pass also propagates the callee's *postconditions* into the caller's set, gated on a termination flag the inferrer maintains for each method. A postcondition of a non-terminating callee describes a state that is established only *if* the callee returns, so propagating it unconditionally would be unsound when the callee may not terminate. The pass therefore propagates `n`'s postconditions into `m` only when `n.terminates()` is true. The polymorphic-dispatch case is handled identically to preconditions, with the disjunction over candidates.

D. What the algorithm is and is not

The algorithm is a *lifting* step over an existing single-pass result, not a complete WP transformer. It does not:

- compute the weakest precondition through arithmetic or array statements (it accepts the inferrer's clauses verbatim);
- handle aliasing (the inferrer's substitution is syntactic);
- lift through loops (the inferrer's loop-invariant heuristic handles this in the isolated pass);
- discharge clauses against an SMT solver (the inferrer's downstream OpenJML pipeline handles this).

It does enough — branch-guard lifting and polymorphic-dispatch disjunction — to expose the structural classes of additions that a single-pass propagation strategy cannot produce. Section V measures whether those additions are empirically substantial on real code.

IV. IMPLEMENTATION

The pass is implemented as a single Java class, `com.jml.inferrer.analysis.CompositionalAnalyzer`, sitting next to the existing `InterproceduralAnalyzer` in the inferrer's module. The class is approximately 410 lines and depends only on the inferrer's existing model (`MethodSpecification`, `SpecificationCache`, `CallGraph`) and `JavaParser`.

A. Entry point

The pass is invoked after the inferrer's isolated pass populates the cache:

```
SpecificationCache cache = new SpecificationCache();
CallGraph callGraph = new CallGraphBuilder()
    .buildFromCompilationUnits(units);
MethodSpecificationInferencer inferencer =
    new MethodSpecificationInferencer(cache,
        callGraph);
for (MethodDeclaration m : allMethods) {
```

```

    cache.put(signatureOf(m),
              inferer.inferSpecification(m));
}

// Compositional refinement (this paper):
CompositionalAnalyzer.RefinementResult result =
    new CompositionalAnalyzer(cache, callGraph,
                              declarations)
        .refineAll();
'''
}

```

The `RefinementResult` carries the number of methods refined, total iterations across all SCCs, and the SCC count. Callers can choose whether to invoke the pass — the design is opt-in so a deployment that prefers the cheaper single-pass output can omit it.

B. Two real bugs surfaced by the corpus-scale run

The analyser had been validated by a 15-test unit suite before this study. Running it on Apache Commons Lang surfaced two bugs that the unit tests did not exercise.

Bug 1: `lastIndexOf('.')` in `polymorphicCandidates`. The method searches for sibling cache entries by splitting each signature into class name and method name. The split used `sig.lastIndexOf('.')` — which returns the position of the *rightmost* dot. For a signature like `ArchUtils.getProcessor(java.lang.String)` the rightmost dot is the one inside the parameter type `java.lang.String`, not the separator between class and method. The resulting substring slice threw `StringIndexOutOfBoundsException` with a negative-length range. The fix is to use `lastIndexOf('.', paren - 1)`, which restricts the search to before the parameter-list opening paren.

Bug 2: same pattern in `resolveCallee`. The same algorithm and the same bug appeared in the call-resolution code. Fixed identically.

Both bugs were silent under the unit-test inputs (which used short parameter types) and only manifested on the corpus run with fully-qualified parameter types from JavaParser. We report them because they illustrate a general risk for analysers that work on JavaParser ASTs: the printed representation of a type is implementation-dependent, and any string-splitting code over signatures must consider the worst case.

C. Repository layout

The analyser is at `src/main/java/com/jml/inferrer/CompositionalAnalyzer.java` in the project tree. Unit tests at `src/test/java/com/jml/inferrer/analysis/CompositionalAnalyzerTest.java` (15 tests, all passing after the fixes). The measurement harness for this paper is at `src/test/java/com/jml/inferrer/analysis/CompositionalAnalyzerCommonsLangExperimentTest.java`.

Raw output is committed at `journal/rq3_composition_measurement.txt`.

V. EMPIRICAL MEASUREMENT

A. Subjects and method

We measure on the source jars of Apache Commons Lang 3.14.0 (246 source files, 3,494 methods produce a non-empty isolated spec) and Apache Commons IO 2.13.0 (245 source files, 2,111 methods produce a non-empty isolated spec). These are the same libraries used by the predecessor work [1], [2]; using them lets cross-paper comparisons proceed without confounding from corpus differences. The Guava library, which the predecessor papers also cover, is excluded from the present measurement: its source-jar size pushes the JavaParser-driven harness into a deep recursion that overflows the default JVM stack (Section VII).

For each library, the harness:

- 1) Parses every `.java` entry in the sources jar.
- 2) Builds the call graph (`CallGraphBuilder.buildFromCompilationUnits`).
- 3) Runs the existing `MethodSpecificationInferer.inferSpec` on every method declaration. The inferer's internal `InterproceduralAnalyzer` runs as part of this pass, propagating callee preconditions in the standard single-pass manner. The resulting per-method precondition and postcondition counts are snapshotted as the *isolated baseline*.
- 4) Invokes `CompositionalAnalyzer.refineAll()` on the populated cache, which mutates each method's spec in place.
- 5) For each method, diffs the post-refinement spec against the baseline and counts newly-added clauses. Each new precondition is categorised by syntactic shape (null check, range/bound, branch-conditional implication, disjunctive, other) for the RQ2 distribution.

All measurements are deterministic given the inputs; we report each exactly once. The harness, the per-library output, and the configuration knobs are committed to the replication package.

B. Results

Table I reports the headline measurements.

RQ1 is answered: the second pass strictly extends the isolated set on both libraries. On Commons Lang, 43.85% of methods in the cache receive at least one new clause from the refinement; on Commons IO, 36.33% do. In total the pass adds 7,764 new precondition clauses and 7,221 new postcondition clauses across the two libraries. The single-pass propagation is *not* subsuming. The result holds even though Commons Lang's `InterproceduralAnalyzer` pass was already doing single-pass callee-precondition propagation during isolated inference; the second pass discovers obligations the first pass cannot express.

TABLE I
EFFECT OF COMPOSITIONAL REFINEMENT OVER THE ISOLATED
SINGLE-PASS BASELINE. *Refined* COUNTS METHODS WHOSE SPEC GAINED
AT LEAST ONE NEW CLAUSE.

	Commons Lang	Commons IO
Methods in cache	3,494	2,111
Methods refined	1,532	767
as % of cache	43.85%	36.33%
Methods with new precondition	1,160	522
Methods with new postcondition	1,270	666
New precondition clauses	5,763	2,001
avg per refined method	4.97	3.83
max on one method	132	41
New postcondition clauses	5,206	2,015

C. Shape distribution

Table II categorises the newly-added preconditions by syntactic shape. The categorisation runs on each new clause independently and is structurally exhaustive (every clause falls in exactly one bucket; ties resolve in favour of structural shape over content, so a clause containing `==>` is counted as branch-conditional even if it also mentions `!= null`).

TABLE II
SYNTACTIC SHAPE OF THE NEWLY-ADDED PRECONDITIONS.
BRANCH-CONDITIONAL IMPLICATIONS DOMINATE ON BOTH LIBRARIES,
REFLECTING THAT THE SECOND PASS WALKS THE CALLER’S
CONTROL-FLOW CONTEXT THAT THE SINGLE PASS DOES NOT.

Shape	Commons Lang		Commons IO	
	count	%	count	%
Branch-conditional (<code>==></code>)	3,029	52.6%	857	42.8%
Null check (<code>!= null / == null</code>)	1,255	21.8%	695	34.7%
Range / bound (<code><, <=, >, >=</code>)	1,143	19.8%	378	18.9%
Disjunctive (<code> </code>)	248	4.3%	36	1.8%
Other	88	1.5%	35	1.7%
Total new preconditions	5,763	100%	2,001	100%

RQ2 is answered: branch-conditional implications dominate, and they are the shape the single pass structurally cannot produce. On Commons Lang, 52.6% of all newly-added preconditions are `guard ==> obligation` clauses; on Commons IO, 42.8%. These clauses arise from calls inside an `if` or ternary branch whose enclosing guard must be lifted into the precondition. A single-pass propagation, which appends the callee’s substituted preconditions *unconditionally*, can produce the obligation part but cannot synthesise the `guard ==>` prefix because it has no access to the caller’s control-flow context at the point of propagation.

Null checks (22–35%) and range/bound clauses (19%) make up most of the remainder. These are clauses the single pass *would* produce when the call site is unguarded; the second pass adds them when the call site is guarded by a condition the single pass would not lift. Disjunctive clauses (1.8–4.3%) come from polymorphic dispatch sites where multiple subtype callees contribute alternative obligations; the residual *other*

(1.5–1.7%) is the long-tail shape that includes arithmetic comparisons not matched by the range/bound regex.

D. Runtime cost

The second pass completes in 3.5 seconds on Commons Lang (3,494 methods, 6,223 SCCs visited, all singletons) and proportionally on Commons IO (2,111 methods, 9 seconds for both runs combined including parsing and the baseline isolated pass). No SCC hit the fixpoint cap; the call graphs of both libraries are acyclic under the analyser’s signature-matching scheme. The refinement step is therefore an addition of seconds on top of an inference pass that already takes minutes on libraries of this size; it is a clearly affordable enhancement.

RQ3 is answered: the second pass is cheap and fixpoint iteration never fires at the libraries’ scale. The runtime characteristic is dominated by the AST walk per method (`decl.findAll(MethodCallExpr.class)`), which is linear in the body size; the lifting and substitution steps add constant factors per call site. The cost scales with the number of call sites in the corpus (7,719 on Commons Lang, 4,213 on Commons IO).

VI. DISCUSSION

A. The single-pass strategy is not subsuming

The measurement settles a question the predecessor papers left open: the isolated-pass inferer’s `InterproceduralAnalyzer` [1] does not capture what a compositional refinement would. Two structural shapes are out of reach by construction: branch-conditional implications that depend on the caller’s control-flow context, and disjunctive obligations from polymorphic dispatch. The data shows these are not edge cases — branch-conditional implications make up the *majority* of newly-added preconditions on both libraries.

The implication for the inferer’s downstream consumers (test generators, IDEs, LLMs) is that running the second pass before emission delivers a measurably richer spec. Whether the additional clauses translate into stronger downstream behaviour (more bugs caught, fewer false positives in static analysis, better test inputs) is the empirical question for follow-up work; we report only that the gap exists and that it is structurally explainable.

B. Why the single pass already does so much

It is also worth noting what the single pass *does* cover. The isolated baseline already produces 11.95% bytecode overhead under synthetic specs and 10.07% under real inferred specs [2]; a substantial fraction of those clauses are interprocedural propagations the `InterproceduralAnalyzer` produces during isolated inference. The second pass adds 5,763 *new* clauses on top of an already-rich baseline on Commons Lang. The single pass does the right thing on

unguarded call sites and on calls to single-candidate methods; it is the guarded and polymorphic cases where the gap appears.

C. Implications for analyser architecture

For inferers that target downstream tools where richer specs help (verification, test generation, LLM consumption), the second pass is clearly worth integrating: an extra 3.5 seconds on Commons Lang for 43.85% more methods receiving refined specifications is a strong trade. For inferers that target downstream tools that consume specs eagerly (auto-completion, hover text in an IDE), the runtime cost may not justify the extra clauses; the second pass is opt-in by design so employers can choose.

A further architectural implication: an inferer that wants to keep its pipeline single-pass can still capture some of the compositional value by enriching its `InterproceduralAnalyzer`'s single-call-site lifting. The branch-conditional shape, which dominates the gap, requires the analyser to know the enclosing guard of each call site. That information is available during the AST walk that the isolated pass already does; the single-pass analyser could in principle lift its substituted clause through the guard before adding it. The polymorphic-dispatch shape is harder to incorporate into a single-pass analysis because it requires looking up sibling cache entries at the point of propagation, which is what the second pass does anyway. Whether the single pass should be extended to handle the easy half of the gap, or whether the two-pass architecture is the right factoring, is a design choice the present measurement makes explicit.

D. Limitations

The 414-line analyser is a scaffold, not a complete WP transformer. It does not lift through loops (the inferer's loop-invariant analyser handles those in the isolated pass), does not reason about aliasing (the syntactic substitution treats two distinct names as distinct values), and does not interact with the SMT verifier that discharges the final clauses. These are the boundaries of the present contribution; the comprehensive WP transformer would be a substantially larger development.

The 132 maximum new preconditions on one Commons Lang method is the worst case we observed. The method in question is a deeply branching configuration loader; its 132 new clauses are mostly branch-conditional implications gating different code paths. Whether the resulting spec is *useful* to a downstream tool — i.e. whether it is parsimonious enough to be readable and tight enough to be informative — is the question we defer to the downstream-impact study planned as future work.

VII. THREATS TO VALIDITY

a) *Internal validity.*: The measurement reports per-method clause-count diffs. These are deterministic given the inputs: rerunning the harness produces byte-identical metric output (we verified). The 5,763 / 2,001 new-precondition counts are therefore exact for the inputs reported. The shape

categorisation uses a simple regex-based bucketing; we documented its order-of-precedence (structural shape over content) so the buckets are reproducible. A small number of clauses in the *other* bucket (1.5–1.7%) are arithmetic comparisons our regex does not match (e.g. `abs(x) <= K`); their existence does not affect the headline result.

b) *External validity.*: The measurement is on two libraries (Commons Lang and Commons IO), not on a broader corpus. Generalisation to less utility-shaped code (application logic, framework code, generics-heavy collections) is left to follow-up. The exclusion of Guava is forced by a stack-depth limit in the JavaParser-driven harness we use to build the specification cache (the harness recurses on large compilation units; raising `-Xss` would let Guava run, but our default configuration is the realistic one for typical Maven test invocations). The fix is mechanical (either raise the stack size or rewrite the call-graph builder iteratively); we do not expect Guava to qualitatively change the conclusion, because the structural classes of additions (branch-conditional, polymorphic-dispatch) are properties of the language, not the library.

The single-pass baseline against which we compare is the inferer's `InterproceduralAnalyzer` from [1]. A different single-pass implementation might propagate more information per call site (e.g. guarded propagation in a single pass), narrowing the gap. We do not claim the second pass is strictly necessary against all conceivable single-pass strategies; we claim it materially extends the specific baseline of an established prior system.

c) *Construct validity.*: “Refined” is operationalised as “the spec gained at least one clause that was not in the baseline”, using exact string equality on clause text. Two clauses that are semantically equivalent but syntactically different (e.g. `a < b` vs. `b > a`) would count as additions even if redundant. The analyser does not normalise clauses before equality, so the new-clause count is an upper bound on the number of genuinely new *logical* obligations. The shape categorisation is unaffected: a duplicate `a < b` is still a range/bound clause.

The callee-resolution step uses simple-name matching, which can return a wrong callee for overloaded methods. The `substituteArguments` step then rejects the substitution if the formal and actual arity disagree; in our run, 4,631 of 7,719 Commons Lang call sites pass the arity check (60%). The remaining 40% are either inferer-internal calls without a cached callee or name collisions with the wrong callee. A full type-resolving harness would close this measurement gap, but the affected call sites would either add additional clauses (strengthening the headline result) or be rejected by arity (no effect on the result). The reported numbers are therefore a *lower bound* on the true compositional gain.

d) *Conclusion validity.*: The empirical claims here are exact measurements, not statistical inferences; no significance test is meaningful. The conclusions (“the second pass refines 43.85% of methods on Commons Lang”,

“branch-conditional implications dominate the additions”) hold for the specific corpora reported and for the specific `CompositionalAnalyzer` implementation in the artefact. We do not claim that the same percentages would obtain on other libraries or on a different second-pass implementation; the structural explanation (single-pass propagation cannot lift through guards or dispatch) suggests the qualitative finding will transfer, but quantitative transfer is an open question.

VIII. RELATED WORK

a) Compositional reasoning frameworks.: The classic foundations are Hoare’s axiomatic semantics [4] and Dijkstra’s weakest-precondition calculus [3], [5]. Cousot & Cousot’s abstract interpretation [6] provides the general framework within which our second-pass refinement sits: the isolated pass is a sound but conservative approximation of the fixpoint; our refinement narrows it via callee summaries. Boogie’s verification-condition generator [7] performs the full WP transformation that our scaffold approximates; Boogie targets verification rather than the cheap-and-rich JML emission this work serves.

b) Procedure summaries and interprocedural analysis.: Calcagno et al.’s Infer [8] computes compositional summaries via bi-abduction in separation logic; the architecture is genuinely compositional from the outset and does not need a “second pass” because the first pass is already compositional. Houdini [9] conjectures candidate annotations and prunes them via ESC/Java calls; the loop is closer to test-and-refute than to constructive compositional inference but the spirit is similar. Daikon’s likely invariants [10] are observed dynamically per program point and do not have an analogue of a second pass; the question we address (does single-pass propagation capture compositional information) is specific to static syntactic inferers.

c) Specification inference for JML.: The immediate predecessor is [1], which introduced the heuristic JML inferer we extend, and [2], which addressed the distribution problem (embedding inferred specs in compiled bytecode and OpenAPI descriptions). PreInfer [11] uses dynamic symbolic execution to infer preconditions; its propagation strategy is trace-driven rather than syntactic and is not directly comparable. Jdoctor [12] extracts exception preconditions from Javadoc; its scope is narrower than ours but its precondition synthesis is purely textual and does not propagate.

d) Polymorphic dispatch in static analysis.: Tip et al. [13] address polymorphic dispatch via class-hierarchy analysis (CHA) and rapid-type analysis (RTA), the techniques the inferer’s `CallGraph` uses to enumerate candidate callees. Our disjunction-over-candidates is the static equivalent of summarising the runtime-may-dispatch set. The trade-off is the standard CHA precision/conservatism trade-off: disjunction over a large set of candidates produces preconditions that are easy to satisfy and therefore less informative.

e) Mutation testing as a downstream evaluation.: PIT [14] is the mutation-testing pipeline the predecessor work

uses to evaluate downstream spec quality. The present article does not run a mutation experiment; we report only the structural-extension claim. The follow-up evaluation will run PIT against test suites generated from the isolated baseline and from the compositionally-refined specs to ask whether the additional clauses translate into more bugs caught at the suite level.

IX. CONCLUSION

We asked whether a single-pass callee-precondition propagation, the strategy used by an established heuristic JML inferer, captures everything a top-down compositional refinement would. We implemented the refinement as a 410-line analyser and measured its effect on two real-world libraries. The single pass does *not* subsume: compositional refinement adds at least one new clause to 43.85% of Commons Lang methods and 36.33% of Commons IO methods, for a total of 7,764 new precondition clauses and 7,221 new postcondition clauses across the two corpora.

The shape distribution explains the gap. Branch-conditional implications (`guard ==> obligation`) make up 53% of the additions on Commons Lang and 43% on Commons IO. These clauses arise from calls inside branch bodies whose enclosing guard must be lifted into the caller’s precondition. A single-pass analyser that appends the callee’s substituted precondition unconditionally cannot produce the `guard ==>` prefix; the WP-style walk through the body that the second pass performs is what supplies the missing context. Null checks and range bounds account for most of the remainder; polymorphic-dispatch disjunctions account for a small but visible 2–4%.

The runtime cost of the second pass is small: 3.5 seconds on a 3,494-method library, with no fixpoint iteration triggered at the libraries’ scale. The refactoring is therefore a clear value: an inferer-pipeline addition of seconds yields a measurably richer spec set for downstream consumers.

What this paper does not measure is whether the additional clauses materially improve downstream tooling — whether tests generated from the refined specs catch more bugs, whether IDEs displaying the refined contracts help reviewers, whether LLMs reading them produce better completions. Those questions are the planned next stage of this thesis programme; the present article establishes the structural prerequisite, that the additional clauses exist, before the downstream-impact question can be meaningfully posed.

Two bugs were uncovered in the analyser by the corpus-scale run (both string-splitting errors that the unit-test inputs did not exercise); we report them as a methodological note. The analyser ships with the corpus-scale measurement harness as a runnable replication artefact.

DATA AVAILABILITY

A replication package containing the `CompositionalAnalyzer` source, the harness that

produced every measurement reported here, the per-library output logs, and the parsed clause samples is available at [https://github.com/\[anonymized\]/jml-inferrer](https://github.com/[anonymized]/jml-inferrer).

REFERENCES

- [1] B. Edmonds and M. Utting, “Do specifications help LLMs write better unit tests? an empirical study of specification-guided test generation with heuristically inferred JML,” 2026, manuscript under review at the Journal of Software: Evolution and Process. Replication package: <https://github.com/ziggyfish/jml-inferrer>.
- [2] —, “Embedding inferred JML contracts in Java bytecode and OpenAPI specifications,” 2026, manuscript under submission to SANER 2027. Replication package: <https://github.com/ziggyfish/jml-inferrer>.
- [3] E. W. Dijkstra, “Guarded commands, nondeterminacy and formal derivation of programs,” *Communications of the ACM*, vol. 18, no. 8, pp. 453–457, 1975.
- [4] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [5] E. W. Dijkstra, *A Discipline of Programming*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1976.
- [6] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints,” in *ACM Symposium on Principles of Programming Languages (POPL)*, 1977, pp. 238–252.
- [7] M. Barnett and K. R. M. Leino, “Weakest-precondition of unstructured programs,” in *Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, 2005, pp. 82–87.
- [8] C. Calcagno, D. Distefano, J. Dubreil, D. Gabi, P. Hooimeijer, M. Luca, P. O’Hearn, I. Papakonstantinou, J. Purbrick, and D. Rodriguez, “Moving fast with software verification,” in *NASA Formal Methods Symposium (NFM)*, 2015, pp. 3–11.
- [9] C. Flanagan and K. R. M. Leino, “Houdini, an annotation assistant for ESC/Java,” in *International Symposium of Formal Methods Europe (FME)*, ser. LNCS, vol. 2021. Springer, 2001, pp. 500–517.
- [10] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, “The Daikon system for dynamic detection of likely invariants,” *Science of Computer Programming*, vol. 69, no. 1–3, pp. 35–45, 2007.
- [11] A. Astorga, S. Srisakaokul, X. Xiao, and T. Xie, “PreInfer: Automatic inference of preconditions via symbolic analysis,” in *48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2018, pp. 678–689.
- [12] A. Blasi, A. Goffi, K. Kuznetsov, A. Gorla, M. D. Ernst, M. Pezzè, and S. D. Castellanos, “Translating code comments to procedure specifications,” in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA ’18. ACM, Jul. 2018, pp. 242–253. [Online]. Available: <http://dx.doi.org/10.1145/3213846.3213872>
- [13] F. Tip, P. F. Sweeney, C. Laffra, A. Eisma, and D. Streeter, “Practical extraction techniques for Java,” *ACM Transactions on Programming Languages and Systems*, vol. 24, no. 6, pp. 625–666, 2002.
- [14] H. Coles, “PIT: Mutation testing system for Java,” <https://pitest.org>, 2016, accessed: 2025-01-15.